Structural Generalization in Autonomous Cyber Incident Response with Message-Passing Neural Networks and Reinforcement Learning

Jakob Nyberg
Division of Network and Systems Engineering
KTH Royal Institute of Technology
Stockholm, Sweden
jaknyb (at) kth.se

Pontus Johnson
Division of Network and Systems Engineering
KTH Royal Institute of Technology
Stockholm, Sweden
pontusj (at) kth.se

2024 IEEE International Conference on Cyber Security and Resilience (CSR) | 979-8-3503-7536-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/CSR61664.2024.10679456

***Abstract*—We believe that agents for automated incident response based on machine learning need to handle changes in network structure. Computer networks are dynamic, and can naturally change in structure over time. Retraining agents for small network changes costs time and energy. We attempt to address this issue with an existing method of relational agent learning, where the relations between objects are assumed to remain consistent across problem instances. The state of the computer network is represented as a relational graph and encoded through a message passing neural network. The message passing neural network and an agent policy using the encoding are optimized end-to-end using reinforcement learning. We evaluate the approach on the second instance of the Cyber Autonomy Gym for Experimentation (CAGE 2), a cyber incident simulator that simulates attacks on an enterprise network. We create variants of the original network with different numbers of hosts and agents are tested without additional training on them. Our results show that agents using relational information are able to find solutions despite changes to the network, and can perform optimally in some instances. Agents using the default vector state representation perform better, but need to be specially trained on each network variant, demonstrating a trade-off between specialization and generalization.**

***Index Terms*—cyber security, reinforcement learning, graph learning, relational learning, generalization**

I. Introduction

We focus on the automation of cyber security incident response, where we imagine an automated *agent* that takes actions and modifies the computer network to address ongoing incidents. An example scenario is ransomware spreading in a corporate network, where action is required to prevent further damage. Following previous work in this domain [1]–[6], we frame incident response as a stochastic decision problem, or a game between agents defending the network and agents that attempt to compromise it. A common method used to find a solution to this class of problem is reinforcement learning (RL) [7], which has been applied in the context of incident response by several authors [8], [9]. rl assumes the dynamics of the problem is not known, which is common for real-world problems, but that the agent can receive observations of the problem state and affect the state through a set of actions.

Computer networks are often dynamic in structure, with hosts connecting and disconnecting over time. We consider it an important feature that a network management agent can handle changes to the network with no additional training, so called *zero-shot generalization*. There are several proposed methods for facilitating zero-shot generalization in reinforcement learning [10]. One approach is *relational* reinforcement learning, where knowledge about relations between objects in a domain is assumed to be transferable across problem instances [11].

We use an approach for relational reinforcement learning proposed by [12] and adapt it to a network security problem. The state of the computer network is represented as a relational graph, which is encoded with a message-passing neural network (MPNN). An agent then uses this encoding to take actions on objects represented in the graph. We also evaluate a modified version of the approach where the agent only incorporates information from the local neighborhood of the graph. We evaluate the MPNN agent on the second iteration of the Cyber Autonomy Gym for Experimentation (CAGE), a two-player cyber security incident simulator. Previous work has noted that agents trained in this environment have issues with structural generalization [6]. We create a number of variants of the original network topology used in CAGE 2 and use these to test the generalization capabilities of agents. Our results demonstrate that an agent which encodes the network state using an MPNN can generalize with no additional training on these variants, something an agent using a multi-layer perceptron (MLP) encoder and vector representations can not.

II. Background

A. CAGE and CybORG

The Cyber Autonomy Gym for Experimentation (CAGE) is a collection of four cyber security scenario simulators developed by the Technical Cooperation Program (TTCP). All

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

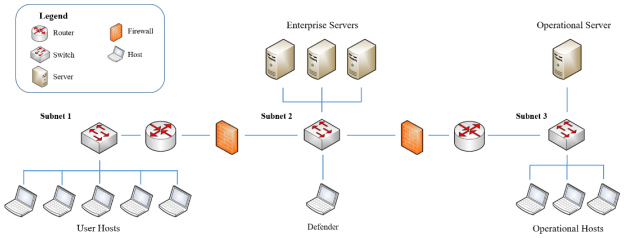


Fig. 1. Image of computer network simulated in CAGE 2 presented on the developers GitHub repository. During a match, the red team starts from an user host and attempts to reach the operational server. This can be achieved by first compromising one of the enterprise servers, which can communicate with the operational machines.

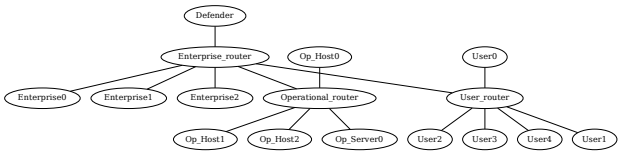


Fig. 2. Graph representation of network structure used in CAGE 2.

were released as part of public competitions to facilitate the development of automated cyber operation agents. The CAGE scenarios share a common underlying simulator infrastructure, the Cyber Operations Research Gym (CybORG) [13]. Although initially described as a simulator where scenarios could be replicated in an emulator of virtual machines, the publicly released portions of CybORG consists of only the simulator. The developers have confirmed that development of the emulator has ceased. The first two scenarios are set in a corporate network and use the same network topology, with the second scenario adding more actions for the red and blue agents. The third CAGE scenario consists of a network of aerial drones under attack. The fourth and most recent scenario returns to the network defense domain, but in a multiagent context. We focus on the second scenario, which we will refer to it as CAGE 2[1].

The scenario premise of CAGE 2 is that a computer network in a manufacturing plant is under attack by a hostile entity. There are two teams acting within the network: A "red" team, consisting of an automated agent, attempts to gain control of machines in the network, with the ultimate goal of reaching the operational server; A "blue" team of network operators attempts to stop the red team from compromising hosts in the network. The network consists of 13 hosts divided into three subnets, separated by firewalls that only allow traffic between certain subnets. An image from the CAGE 2 GitHub repository showing the computer network is presented in 1. A graph representation of the network is shown in Figure 2, with host names as nodes and edges representing connections between them. A third, "green", team is included in the scenario description, representing normal users causing false alerts. However, it is not used in the challenge evaluation, nor in previous work as far as we are aware.

The blue team can act in the network using 13 commands. Ten commands take a host descriptor as a singular parameter, and two actions use no parameters. These are the commands "Sleep" and "Monitor", which function as *NoOp* actions. The "Remove" command will remove red team access from a specified host. The red team can gain two levels of authority on a host. If the red team has reached the second level, "Remove" will not work and access can only be removed by the "Restore" command. The command "Analyze" returns a list of processes and files on a specified host. This can inform the blue team that the red team has gained access to the host. The blue team can use eight "Decoy" commands[2]. These create processes on a specified host that the red team will always fail to exploit.

The observed state provided to both teams is a mapping from host names to lists of simulated processes, network connections and files. CAGE 2 also provides a wrapper function that encodes the mapping into a single vector. The wrapper function applies a set of heuristics to the host data, and provides two categorical values per host. One indicates a type of activity that has been observed on the host, which is based on port usage. The other shows what level of access privileges the red agent has obtained on the host. This is based on the number of open connections, and the list of files on the host. The wrapper function encodes the categorical values to binary, making the state vector $4 \cdot 13 = 52$ elements long. In addition to simplifying the observation space, the wrapper flattens the parametric action space. This gives the blue team $2 + (13 \cdot 11) = 145$ actions. All related works we are aware of have used the wrapped observation and action spaces for blue team agents. There is no defined terminal state for the simulation. The evaluation used for the challenge truncates each round at one of 30, 50 or 100 steps. During an ongoing game, the red team can continually keep attempting to compromise hosts as they can never be fully removed from the network. CAGE 2 includes two heuristic agents to act as the red team, named "B-Line" and "Meander". "Meander" operates similar to a breadth-first search, where it attempts to gain access to all machines in a subnet before moving on to the next. "B-line" selects hosts from a predefined list of hosts, which constitute the shortest path between the attacker starting point and the operational server.

Scoring for the blue team is based on both red team actions and their own actions. When the red team gains access to a system, the blue team receives a penalty. The size of the penalty is proportional to the importance of the host to the organization, with the operational server being the most important. The blue team also receives a penalty each time the "Restore" action is performed on a host. As part of the challenge, CAGE 2 uses a standardized evaluation scheme. Agents are evaluated against both the B-Line and Meander agents on 1000 episodes

[1]www.github.com/cage-challenge/cage-challenge-2

[2]There is a discrepancy between the code and the documentation. The code contains eight decoys, but the documentation only lists seven. The "Sleep" command is not listed either.

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

with lengths of 30, 50 and 100 steps. Average rewards are calculated for each pairing, and a total score is calculated as the sum of average rewards. The highest possible score for is 0.

B. Learning from Graphs

Graphs are data structures that represent relationships or interactions between objects or entities. In a simple formulation, a graph consists of a set of *nodes* $\mathcal{V} = \{v_1, \ldots, v_n\}$, representing objects, and *edges* $\mathcal{E} \subseteq \{(u, v) \mid (u, v) \in \mathcal{V}^2\}$ representing relations between objects. Many functions and algorithms have been proposed to analyze aspects of graph-structured data [14]. We focus on *message-passing* algorithms, where node representations are at every iteration updated using an aggregation of *messages* from their immediate neighborhood. A general message passing update rule is

$$h_v^l = \text{UPDATE}(\text{AGGREGATE}(v), h_v^{l-1}) \tag{1}$$

where AGGREGATE is a permutation invariant aggregation function, and UPDATE is a function of the aggregated neighborhood of v and the node feature at step l. A graph representations can be obtained through an aggregation of the node representations. As long as all aggregations are permutation invariant, message passing algorithms create representations of the graph that are permutation invariant and equivariant [14, Ch. 5].

The update and aggregation functions of the message-passing function in Equation 1 can be implemented using neural networks, forming as class of methods called message-passing neural network (MPNN). MPNNs shares parameters across nodes, meaning that although the size of an input graph may change, the number of parameters in the model does not necessarily need to. There are several proposed architectures of MPNN, which use different schemes of aggregations and updates. These have different abilities to represent and discriminate graph structures, and some will fail to discriminate certain graph structures [15].

Representations produced by message passing are dependent on the number of iterations the algorithm runs, which is equivalent to the number of layers in an MPNN. This dictates the size of the *receptive field* of each node or, put in another way, how far information travel between nodes in the graph. With too few steps, node features may not be expressive enough to be distinguished. On the other hand, as the number of message-passing steps increase, node representations may become homogenized across the graph, a concept referred to as *over-smoothing*. The optimal number of message passing iterations is dependent on the graph and the downstream task.

The term MPNN is sometimes used interchangeably with graph neural network (GNN). We prefer to use the term MPNN as not all neural networks applied on graphs use message-passing, making GNN a broader descriptor than the methods we use require.

C. Reinforcement Learning

Reinforcement learning is a field of machine learning for solving decision and control problems [7]. Problems solved with reinforcement learning can usually be expressed as a Markov decision process (MDP). An MDP can be described as a tuple of states, actions, rewards and state transition probabilities. The reward function $R : A \times S \to \mathbb{R}$ associates each state and action pair with a real value. For a finite MDP, the goal of an agent is to maximize the expected total reward $G = \sum_t^T R_t$, where T is the step the MDP terminates at.

In each state an agent decides on an action using a *policy*, a function from a state to an action or distribution over actions, $\pi : S \to \Delta(A)$. The *value* of a policy π is denoted as V^π, and is the expected total reward in a state s when following the policy π. An optimal policy π^* is associated with the optimal value function $V^* := \max_\pi V_\pi(s)$. Reinforcement learning methods attempt to approximate the optimal policy or value function through continuous interaction with the problem. Policy gradient reinforcement learning methods optimize the policy function towards the optimal policy through gradient descent. Actor-critic methods extend this method by also estimating the value function as part of the optimization process.

For some problems, decisions can naturally be factorized into a series or set of action variables. This includes the CAGE 2 environment described in Section II-A, where actions are separated into hosts and commands. Given a problem with a joint action $\mathbf{a} = (a_1 \ldots, a_i)$, the probability of taking $\mathbf{a}$ in the state S can be expressed as a product of conditional probabilities,

$$\pi(\mathbf{a}|S) = \prod_i^{|\mathbf{a}|} \pi_i(a_i|S, a_1, \ldots, a_{i-1}) \tag{2}$$

Similar forms of policy factorization has been used in previous works on reinforcement learning [5], [12], [16].

D. Symbolic Relational Deep Reinforcement Learning

We use an architecture for relational learning named Symbolic Relational Deep Reinforcement Learning (SR-DRL) [12]. SR-DRL is intended for problem domains that can be represented using objects and actions are taken on those objects. Objects in the problem and their relations are represented as a relational graph, and an MPNN is used to encode the graph as input to a policy function. Policy factorization is used to separate joint actions into conditional parameters for additional flexibility in representation. SR-DRL generates a whole-graph representation in addition to node representations. The graph representation is updated sequentially after each message-passing step, incorporating the previous representation in the next iteration. The MPNN and policy function are trained end-to-end using reinforcement learning. An action-critic scheme is used when training agents, where the value function estimator is implemented as a neural network taking the graph representation as input.

III. Implementation of MPNN Agents on CAGE 2

To apply the SR-DRL approach on CAGE 2, we reshape the vector observation space into a relational graph. We also modify the action space. An agent has to make two decisions

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

at each turn: a command to run and a host to run it on. This is reformulated as the agent selecting a node in the graph, and then an action based on the node representation. We use proximal policy optimization (PPO) to learn a policy function, but any policy gradient method, such as A2C can be used instead. Code use for training and evaluating agents is available on GitHub[3]

As described in Section II-D, SR-DRL incorporates the full graph representation as part of the local node representations [12]. We hypothesize that including the graph representation in the state is unnecessary for problems where reasoning only requires information from the local neighborhood of a node. To test this, we evaluate a modified agent that removes the graph component of the policy input alongside the original version used for SR-DRL. The resulting agent only incorporates the graph representation to estimate the value function during training. During inference, the agent only uses information from a k-hop neighborhood to act, where k is the number of message-passing steps. Another change that comes from using only local information is the policy factorization. The implementation of SR-DRL factorizes the policy such that the agent first selects an action to take, based on the global graph representation, and then what object to act on. In order to only use the local context, we flip this ordering. The agent thus first selects the object to act on, and then what command to execute on it.

A. Local Message-Passing Scheme

The local message passing scheme is largely unchanged from the original implementation [12], albeit with the graph representation removed from the node update. The update to the representation h for a node v at layer l is thus defined as

$$h_v^l = h_v^{l-1} + \phi_{agg}((v, v_{msg})), \tag{3}$$

where

$$v_{msg} = \max_{u \in \mathcal{N}(n)} \phi_{msg}(u). \tag{4}$$

The graph representation update at layer l is

$$g^l = g^{l-1} + \phi_{glb}\left((g^{l-1}, f(\mathcal{V}))\right) \tag{5}$$

where

$$f(\mathcal{V}) = \sum_{v \in \mathcal{V}} \text{softmax}(\phi_{att}(v)) \cdot \phi_{feat}(v), \tag{6}$$

an attention-based aggregation scheme proposed by [17]. Each function ϕ is implemented as a single-layer neural network, $\phi(x; \mathbf{W}, b) = \rho(x\mathbf{W}^T + b)$, where ρ is a non-linear activation function.

B. Policy Decomposition

The local approach decomposes the policy of the agent into two actions, where the agent first selects a node in the graph and then what action should be taken on it. In our notation, E denotes the size of the representation vectors and N the amount of nodes, $|\mathcal{V}|$. $\mathbf{H}^l_{N\times E} = \{h_1^l, \ldots, h_N^l\}$ is a multiset of node representations after l message passes. $\mathcal{C}$ the set of commands from CAGE 2. We formulate the factorized policy function as

$$\pi(v_i, c_i|\mathbf{H}^l) = \pi_2(c_j|h_i^l, v_i)\pi_1(v_i|\mathbf{H}^l) \tag{7}$$

where

$$\pi_1(\mathcal{V} = v_i|\mathbf{H}^l) = \sigma_i(\{\varphi_1(h_1^l), \ldots, \varphi_1(h_N^l)\}) \tag{8}$$

$$\pi_2(\mathcal{C} = c_j|h_i^l, \mathcal{V} = v_i) = \sigma_j(\varphi_2(h_i^l)) \tag{9}$$

and

$$\sigma_i(x) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}, \tag{10}$$

$\varphi_1 : \mathbb{R}^E \to \mathbb{R}$ and $\varphi_2 : \mathbb{R}^E \to \mathbb{R}^{|\mathcal{C}|}$.

The value function estimation is defined as a function of the graph encoding $g^l_{1\times E}$,

$$V^\pi(g^l) = \varphi_v(g^l) \tag{11}$$

where $\varphi_v : \mathbb{R}^E \to \mathbb{R}$. The critic is only used during training, so the graph representation is not required during inference. Each function φ was implemented as a linear transformation, where $\varphi(x; \mathbf{W}, b) = x\mathbf{W}^T + b$. A schematic showing the computation paths of the various values is shown in Figure 3.

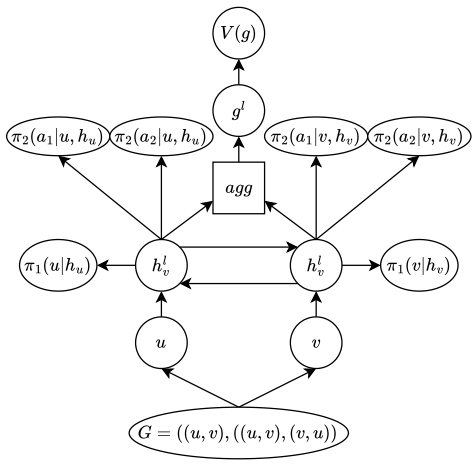


Fig. 3. Schematic showing the computation paths for action probabilities and state value on a graph with two node and two edges.

C. Changes to CAGE 2

We introduced two changes to the CAGE 2 implementation, which we based on commit b5a71c4[4] of CybORG. Our options were to either work from the version of CAGE 2 in the most recent commit of CybORG, or the older version used for the challenge. We opted for the more recent version, as we found it easier to work with.

The first change was reshaping the observation space to a graph, in order to use an MPNN agent. We converted the

[3]www.github.com/kasanari/incident-response-rl-gnn

[4]www.github.com/cage-challenge/CybORG

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

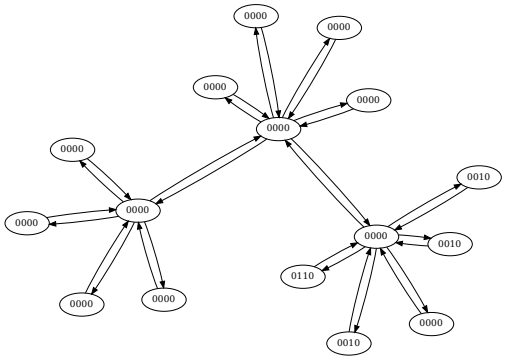


Fig. 4. A state graph at a given point of the CAGE 2 simulation. Each node has two categorical attributes encoded to binary using two bits each. The first attribute indicates the observed activity, and the second the access privilege of the red team agent on that host.

existing CAGE 2 vector state representation into a homogeneous undirected graph. In the graph, each host is represented as a node with a single edge type relating them to other hosts. Each host has two attributes, same as in the original vector, as described in Section II-A. Nodes were connected such that the graph matched the network layout described in the challenge description. Routers were included as nodes, though no observations are associated with them in CAGE 2. They were therefore set to always have no activity, never be compromised and not included in the action space of the blue team. An example of a state graph is shown in Figure 4.

The second change was to the set of allowed actions for the blue team. We removed the host that the red team starts at from the set of hosts that the agent can interact with, and decoy deployment actions were not used. Motivations for these changes are provided in Section VII-B.

IV. Evaluation

Our evaluation of the MPNN agent consisted of training agents in an environment configuration, and then testing the agent on a set of network variants. We only trained against the Meander agent, as it provided more varied observations than the B-Line agent. Previous work on CAGE 2 trained against a combination of attackers, switching between them at random between episodes [6], or switched between models depending on the attacker [4]. In our opinion, access to all forms of red team behavior during training should not be assumed. We thus withheld the B-Line policy during training and used it for evaluation only. The relational learning scheme we employ is not intended to generalize across opponent policies, and the ability to do so is only a fortunate side effect. Episodes were truncated after 50 steps during training. In the absence of blue team interference, this is enough for the Meander agent to be able to reach the operational server.

To evaluate the ability of agents to generalize across structural changes, we created variants of the original network by removing different hosts. Host were selected on the condition that red team agents using both the B-Line and Meander policies could reach the operational host without them. Given this condition, we generated six network variants, each one with an additional randomly selected eligible host removed. This resulted in seven network variants, which includes the original network of 16 hosts. Changing the number of hosts in the network not only changes the observation space of the problem, but also the action space, as the number of machines under the control of the blue team changes. We trained MPNN agents on the network variant with 13 hosts, and evaluate on all variants. This setup gave three larger networks, and three smaller, to evaluate agents on. A specialized MLP agent was trained for each network variant for comparison with the general MPNN agent. We chose an MLP agent for comparison as they have been used in previous work on CAGE 2 [6].

Agents were rated by the scoring scheme described in Section II-A, that test agents with different episode lengths and red teams, as in previous work [4], [6].

The scale of the rewards change depending on the network size and episode length, making comparisons of agents for multiple problem instances difficult. As an alternative performance metric, we calculated the percentage of episodes where the agent did not receive *any* negative reward, which we call a "perfect round". A perfect round thus means that the blue team managed to stop the red team without receiving a penalty.

Agent training and evaluation was done on a machine equipped with 32 GB of RAM, an Intel Xeon Silver CPU with 24 1GHz cores and an NVIDIA Quadro RTX 4000 GPU. Training is also possible on more limited hardware, such as a laptop equipped with 16 GB of RAM and an 11th Gen. Intel i7 CPU. Our code implementation relies on PyTorch Geometric for the message-passing algorithm. We used proximal policy optimization to train agents, as was also done by [6]. The PPO and MLP implementations were sourced from Stable Baselines 3. The code for the MPNN policy is based on the implementation by [18], who in turn ported the code from the original authors [12] to work with Stable Baselines 3. Different selections of hyperparameters were not extensively evaluated, but we present results for MPNN agents with different numbers of layers. We do not include MPNN agents with a layer depth of one, as these were significantly worse than the layer variants. All agents were trained using 500 000 steps of each environment in total.

V. Evaluation Results

An immediate difference between agents using MLP and MPNN models was the training time. With our implementation, MPNN training was roughly 10 times slower than MLP training. In concrete numbers, training each MLP model took roughly 4 minutes, and each MPNN model 40 minutes. The difference was alleviated, however, by the fact that we trained less MPNN agents overall, as we did not have to train a separate agent for every scenario variant.

We calculated average rewards and the total score following the challenge rules, meaning that each combination of attacker, episode length and agent was evaluated for 1000 episodes. The total score presented in Table II is the sum of average rewards

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

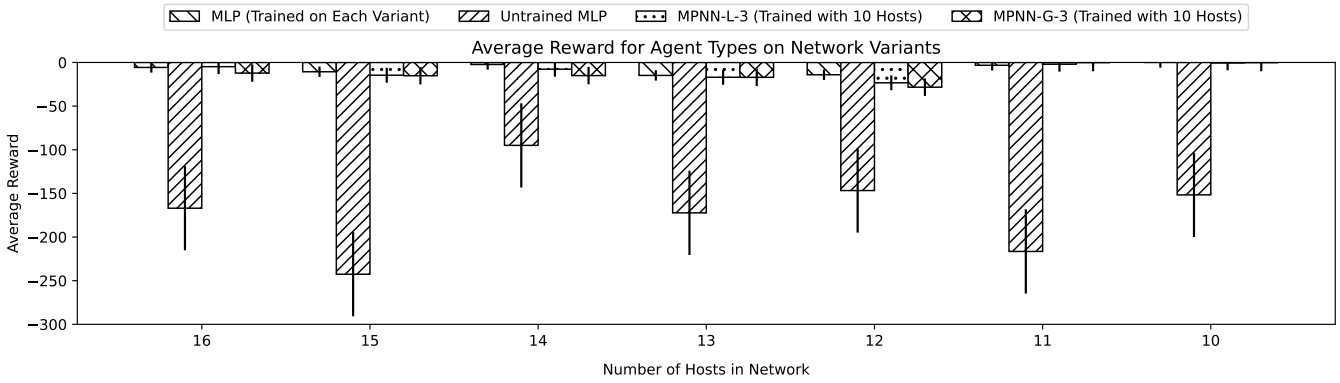


Fig. 5. Bar chart showing rewards for trained and untrained agents averaged over 1000 episodes. MLP agents were specially trained on each network variant. MPNN agents were only trained on the network variant with 13 hosts, and evaluated on others. Agents were trained and evaluated against the Meander red team policy, with episode lengths of 50 steps.

for each configuration. Average rewards for each network variant is shown in Figure 5. We denote the MPNN variants with the shorthand "[GL]-x", where "G" and "L" denote if the model uses global or local representations described in Section III-A, and x denotes the number of layers the model uses. We emphasize that scores are not directly comparable to those of previous work, such as [4] or [6]. The version of CybORG is different, and we omitted actions from the action set that is to the benefit the blue team. We also note that our comparisons are primarily qualitative. The difference between the MPNN models and the MLP is not that the MPNNs performs better or worse at the different scenarios, it is that a trained MLP agent simply *can not function at all* without modification when the observation space changes in size. In general, quantitative differences in rewards between agent variants are not easy to discern with CAGE 2, as has been noted previously [6]. Deviations in scores is high for all agents, relative to the average. We include the scores of an untrained MLP agent, which serve as a baseline zero-shot solution to the problem.

Our primary result is that MPNN agents received scores and percentages of perfect rounds significantly higher than the baseline on network variants not trained on. Scores and the percentages of perfect rounds of MPNN models were lower than those of the trained MLP ensemble, but we note that this is comparing a single agent using one MPNN to an agent using a collection of seven MLP models specially trained on each network variant. L-3 received the highest total score out of the MPNN variants, and L-4 the highest percentage of perfect rounds. The local versions of the MPNN have higher total scores than their global equivalents in aggregate. This lends some credit, albeit not much given the high variance, to our hypothesis that local information is sufficient to find a solution to this problem. The MPNN agents obtain perfect scores on two network variants that the trained MLP agent does not. The untrained MLP agent ensemble never achieves a perfect round, indicating that this can not be done through random action. Table II shows the total scores for all MPNN variants and the MLP ensemble agents, averaged over all seven network variants. Table I shows the percentages of perfects rounds with each network variant.

TABLE I
PERCENTAGES OF EPISODES WITH NO PENALTIES AVERAGED OVER ALL RED TEAM AND EPISODE LENGTH CONFIGURATIONS. MPNN AGENTS WERE ONLY TRAINED WITH 13 HOSTS. MLP AGENTS WERE TRAINED ON EACH NETWORK VARIANT.

Hosts	16	15	14	**13**	12	11	10
MPNN	Episodes With No Penalty (%)						
G-2	5	0	17	0	0	32	0
G-3	1	2	28	23	0	50	74
G-4	0	0	1	0	0	7	1
L-2	12	0	0	0	0	0	0
L-3	17	0	32	18	0	8	49
L-4	10	4	44	34	0	34	89
MLP	Episodes With No Penalty (%)						
Trained	49	0	77	50	0	0	100
Untrained	0	0	0	0	0	0	0

VI. RELATED WORK

There exists several cyber incident simulators with different assumptions[5] and associated solution methods [9]. In the interest of space, we limit works in described in this section to those that use CAGE 2 in some form. We make one exception to this limitation and highlight the work of [3], that combines reinforcement learning, relational graph learning and cyber security similarly to us. They evaluate their approach on another cyber incident simulator, Yawning Titan, which is more abstract than CybORG. Our implementation differs from theirs in that they use a graph embedding algorithm named Feather-G, a method for graph representation based on characteristic functions [19]. It is not explicitly stated in the text that the action space is generalized to different number

[5]github.com/Limmen/awesome-rl-for-cybersecurity

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUM OF AVERAGE REWARDS AND PERCENTAGES OF EPISODES WITH NO PENALTIES, AGGREGATED OVER ALL COMBINATIONS OF NETWORK SIZES, EPISODE LENGTHS AND RED TEAM POLICIES. MPNN MODELS WERE ONLY TRAINED WITH 13 HOSTS. MLP RESULTS ARE AVERAGES FOR SEVEN AGENTS TRAINED ON EACH NETWORK.

Agent	Episodes With No Penalty	Sum Average Reward
MPNN		
G-2	8%	-154 ± 42
G-3	25%	-270 ± 252
G-4	1%	-172 ± 63
L-2	2%	-136 ± 22
L-3	18%	-113 ± 63
L-4	31%	-150 ± 99
MLP (Ensemble)		
Trained	39%	-78 ± 42
Untrained	0%	-2104 ± 605

of hosts. In principle, we could use the node-level version of Feather, Feather-N, to represent nodes as we do with the MPNN.

Our starting point for studying previous work on CAGE 2 are agents that obtained high scores on the challenge. The winners of the challenge, a team from Cardiff University, used an MLP agent trained using PPO in combination with handwritten heuristics. The agent uses a static policy for deploying decoys, derived from a greedy search over the space of possible decoys that can be deployed. The agent also uses hard-coded heuristics to determine which one out of two policies to use in a given episode, one for each red-team policy, and limit the blue team's actions to those that are strictly useful. Although effective in the challenge, this agent formulation can not generalize to unseen configurations, as its scenario-specific heuristics would need to be manually updated. The runner-up on the leaderboard is another MLP agent by a team from the Alan Turing Institute, also trained using PPO. It uses two policy functions with different observation spaces that are switched between by a higher-level policy. The policy used against Meander uses the unmodified observation space, and the one used against B-Line adds information about the number of ports to each host. In terms of the ability to generalize, this is better than the Cardiff approach as it does not hard-code parts of the policy. However, it is still bound to a particular size of the network.

There are works outside the challenge that have used CAGE 2 for development and evaluation. [4] trained agents with different forms of reward shaping on the environment. They base their work on the model by Cardiff University, which in our opinion makes conclusions difficult due to the issues of hard-coding mentioned previously. [20] trains agents on CAGE 2 using a version of deep Q-learning. Similar to us, they create three scenario variants to test their agent, but have to train a separate agent on each version due to their approach being locked to a specific network size. They also note that their approach has issues when introducing additional hosts to the network, but the causes of these issues are not made clear. [6] evaluates a set of neural network models on CAGE 2 on different configurations. The experiments are intended to evaluate attributes that they consider important when agents are applied in real systems, such as inference time and generalization. They note that their agents trained with RL on CAGE 2 are challenged by changes to the network configuration, such as the topology, and differences in the attacker policy.

VII. DISCUSSION

From the results of our experiments, we see that the MPNN agents perform better on the unseen network variants than the untrained MLP baseline. We also see that they can obtain a nonzero percentage of rounds where the red team does not capture any machines, and the blue team does not receive any penalty. We take this as evidence that the MPNN agents are capable of zero-shot generalization in this problem domain. The zero-shot performance is worse than that of MLP policies specifically trained on the network variant. This is in line with results obtained by [12], where the MPNN agents tend to be outperformed by one-shot planners on specific problem instances. The difference in results indicates a trade-off between generalization and specialization. We do not know if a fully generalized policy is possible to find for this problem, but work has been done to analyze this question in other domains [21]. We chose MLP agents as an upper bound of performance, as they are often used in previous work on the CAGE 2 environment. However, these are likely worse than a theoretical optimal policy, meaning that we do not know how close the zero-shot policy is to optimal. We can see from Table I that for some network variants, both the MLP and MPNN agents have few to no perfect rounds.

A. Limits of Relational Learning

An inductive bias of relational reinforcement learning is that the properties and relationships between objects remain constant. This may not be true, depending on the problem domain. In our problem, changing the red team policy will change the underlying dynamics of the problem, and the change in representation that the MPNN introduces does not directly address this. It may indirectly do so if the relational rules learned by the agent is general enough to counter any red team policy, but this is also true for a policy that can be learned by an MLP agent. Other methods of zero-shot learning would need to be applied to address this issue [10].

B. Changes made to CAGE 2

We made a set of changes to the CAGE 2 environment. We removed the host that the red team starts at from the blue team action space. The starting host is different from other hosts in that blue team commands fail when used on it. This is to ensure the red team always has access to the network. If vector observations are used, there is no way to identify the starting host other than by its position in the vector. We chose to not include decoy actions for similar reasons. The decoy actions have specific sets of requirements that need to be fulfilled in order to execute them. This includes ports not being

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.

in use, or the host running a particular operating system. This information is not present in the wrapped observation space, and only the position of the host in the vector encodes this information. In both these cases the position of the host in the state vector acts as an implicit host identifier. This information is lost when the vector is reshaped to a graph, but makes the agent using the graph more general. We can assign a unique label to each node in the graph, which would allow an agent to fully differentiate between objects. However, it also means that the rules learned will be specific to that set of objects and less generally applicable. We could also add the object class to nodes, but then we have to design a fitting ontology. Our opinion is that CAGE 2 is an interesting environment, albeit one that is slightly oversimplified. We believe future work should focus on utilizing the unwrapped observation space, which contains information more in line with what an actual network intrusion detection system would produce.

VIII. Conclusion

We implemented agents for automated network intrusion response that use message-passing neural networks to encode facts about the network. The agents were evaluated using a simulated network environment, the Cyber Autonomy Gym for Experimentation. Our results show that our agents can generalize across network variants without additional training but are outperformed by specially trained policies, indicating a trade-off between generalization and specialization. Our work addresses an issue present in previous work on automated incident response [4], [6]: That agents are bound to a specific size, structure and ordering of the network topology. This is in spite of computer networks being highly variable in structure, and a network operator needs to handle such changes while enacting security policies. We believe that by exploiting relational structure in the problem, agents for cyber incident response can be made more general and reusable. Reusing agents in problems with a different structure, but similar dynamics saves both time and energy, which we consider important for practical use.

References

[1] A. Applebaum, C. Dennler, P. Dwyer, M. Moskowitz, H. Nguyen, N. Nichols, N. Park, P. Rachwalski, F. Rau, A. Webster, and M. Wolk, "Bridging automated to autonomous cyber defense: Foundational analysis of tabular q-learning," in *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, ser. AISec'22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 149–159.

[2] S. Roy, C. Ellis, S. Shiva, D. Dasgupta, V. Shandilya, and Q. Wu, "A survey of game theory as applied to network security," in *2010 43rd Hawaii International Conference on System Sciences*, 2010, pp. 1–10.

[3] J. Collyer, A. Andrew, and D. Hodges, "Acd-g: Enhancing autonomous cyber defense agent generalization through graph embedded network representation." NeurIPS 2022 Workshop, Reinforcement Learning for Real Life, 2022.

[4] E. Bates, V. Mavroudis, and C. Hicks, "Reward shaping for happier autonomous cyber security agents," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, ser. AISec '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 221–232.

[5] K. Hammar and R. Stadler, "Finding effective security strategies through reinforcement learning and self-play," in *2020 16th International Conference on Network and Service Management (CNSM)*. IEEE, 2020, pp. 1–9.

[6] M. Wolk, A. Applebaum, C. Dennler, P. Dwyer, M. Moskowitz, H. Nguyen, N. Nichols, N. Park, P. Rachwalski, F. Rau, and A. Webster, "Beyond cage: Investigating generalization of learned autonomous network defense policies." International Conference on Machine Learning Workshop, ML4Cyber, 2022.

[7] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[8] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2023.

[9] A. M. K. Adawadkar and N. Kulkarni, "Cyber-security and reinforcement learning — a brief survey," *Engineering Applications of Artificial Intelligence*, vol. 114, p. 105116, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197622002512

[10] R. Kirk, A. Zhang, E. Grefenstette, and T. Rocktäschel, "A survey of zero-shot generalisation in deep reinforcement learning," *J. Artif. Intell. Res.*, vol. 76, pp. 201–264, 2023. [Online]. Available: https://doi.org/10.1613/jair.1.14174

[11] S. Dzeroski, L. D. Raedt, and K. Driessens, "Relational reinforcement learning," *Mach. Learn.*, vol. 43, no. 1/2, pp. 7–52, 2001.

[12] J. Janisch, T. Pevný, and V. Lisý, "Symbolic relational deep reinforcement learning based on graph neural networks and autoregressive policy decomposition," 2023.

[13] M. Standen, M. Lucas, D. Bowman, T. J. Richer, J. Kim, and D. Marriott, "Cyborg: A gym for the development of autonomous cyber agents," in *IJCAI-21 1st International Workshop on Adaptive Cyber Defense.*, 2021.

[14] W. L. Hamilton, *Graph Representation Learning*. Springer, 2020.

[15] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. [Online]. Available: https://openreview.net/forum?id=ryGs6iA5Km

[16] O. Vinyals, T. Ewalds, S. Bartunov, P. Georgiev, A. S. Vezhnevets, M. Yeo, A. Makhzani, H. Küttler, J. Agapiou, J. Schrittwieser, J. Quan, S. Gaffney, S. Petersen, K. Simonyan, T. Schaul, H. van Hasselt, D. Silver, T. Lillicrap, K. Calderone, P. Keet, A. Brunasso, D. Lawrence, A. Ekermo, J. Repp, and R. Tsing, "Starcraft ii: A new challenge for reinforcement learning," 2017.

[17] Y. Li, C. Gu, T. Dullien, O. Vinyals, and P. Kohli, "Graph matching networks for learning the similarity of graph structured objects," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 2019, pp. 3835–3845.

[18] A. Chester, M. Dann, F. Zambetta, and J. Thangarajah, "Oracle-sage: Planning ahead in graph-based deep reinforcement learning," in *Machine Learning and Knowledge Discovery in Databases*. Cham: Springer Nature Switzerland, 2023, pp. 52–67.

[19] B. Rozemberczki and R. Sarkar, "Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*. ACM, 2020, pp. 1325–1334.

[20] Z. Zhu, M. Chen, C. Zhu, and Y. Zhu, "Effective defense strategies in network security using improved double dueling deep q-network," *Computers & Security*, vol. 136, p. 103578, 2024.

[21] J. Mao, T. Lozano-Pérez, J. B. Tenenbaum, and L. P. Kaelbling, "What planning problems can A relational neural network solve?" in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/ba90e56a74fd77d0ddec033dc199f0fa-Abstract-Conference.html

Authorized licensed use limited to: Uganda Christian University. Downloaded on October 22,2026 at 22:00:43 UTC from IEEE Xplore. Restrictions apply.